

Skip List
–Destroy List
–Space Complexity Analysis

Program:

```
void destroyList(struct SkipList *lst) {
    // Start at the header
    struct Node *current = lst->header;
    struct Node *next;

    // We only need to traverse level 0 (the "local train" level)
    // because it is the only level guaranteed to contain EVERY node.
    while (current != NULL) {
        // 1. Save the pointer to the next node
        next = current->forward[0];

        // 2. Free the dynamically allocated array of pointers
        free(current->forward);

        // 3. Free the node itself
        free(current);

        // 4. Move to the next node
        current = next;
    }

    // Finally, free the main Skip List structure
    free(lst);
    cout << "Memory successfully freed. Skip List destroyed." << endl;
}

...

case 5:
    cout << "Destroying the current list..." << endl;
    destroyList(list);
    // CRITICAL: We must give the program a fresh, empty list
    // so it doesn't crash if the user tries to insert again.
    break;
```

Space Complexity Analysis

1. Input Space Complexity

Just like the other modules, the input space evaluates the memory of the parameters passed into the function and the overarching data structure it acts upon.

A. Parametric Input Space: $O(1)$

- **The Parameter:** The function signature is `void destroyList(struct SkipList * lst)`. It takes exactly one argument: a memory pointer addressing the main Skip List container.
- **The Math:** This single pointer requires a fixed, microscopic amount of memory (typically 8 bytes) to hold the address.
- **The Conclusion:** Because the size of this parameter never scales up – whether you are destroying a list of 5 nodes or 5 million nodes – the parametric input space is strictly $O(1)$.

B. Full Data Holistic Input Space: $O(n)$

- **The Structure:** The function operates on the fully built Skip List currently sitting in the computer's memory.
- **The Math:** As proven in the general Space Complexity analysis, a Skip List holds n elements using an expected average of $2n$ pointers.
- **The Conclusion:** Because the physical memory footprint of the data structure being destroyed scales directly with the number of elements inside it, the full data holistic input space is $O(n)$.

2. Auxiliary Space Complexity: $O(1)$

Auxiliary space strictly measures the extra, temporary memory the algorithm asks the system for in order to execute its internal logic.

- **The Variables:** *Inside the `destroyList` function, the algorithm declares exactly two local pointer variables:*
`struct Node * current` and `struct Node * next`.
- **No Arrays or Recursion:** *Unlike the Insert and Delete modules, the Destroy module does not need to remember a complex vertical path, so it does not allocate an `update[ ]array`. It simply uses the `next` pointer to remember the address of the upcoming node on **Level 0** before it wipes the `current` node out of existence.*
- **The Conclusion:** *Because the function allocates exactly zero arrays, makes zero recursive calls, and only asks the system for enough memory to hold two temporary tracking pointers, the auxiliary space complexity is a strict, absolute **$O(1)$** .*

Final Total Space Complexity

To find the total space complexity of the module, we add the dominant input space to the auxiliary space:

- **Total Space Complexity** =

Either:

$$\text{Full Data Holistic Space } (O(n)) + \text{Auxiliary Space } (O(1)) = O(n)$$

Or:

$$\begin{aligned} &\text{Input Space}(\text{Full Data Holistic Space } (O(n)) \\ &\quad + \text{Parametric Input Space } (O(1))) + \text{Auxiliary Space } (O(1)) \\ &= O(n) \end{aligned}$$

Algorithm: SkipListDestroy(LST)

1. [Initialize Pointers]

SET CURRENT := HEADER[LST].

Declare a node pointer NEXT.

2. [Traverse Level 0 to Free Nodes]

Repeat while CURRENT != NULL:

// Save the pointer to the next node

SET NEXT := FORWARD[0][CURRENT].

// Free the dynamically allocated array of pointers

CALL free(FORWARD[CURRENT]).

// Free the node itself

CALL free(CURRENT).

// Move to the next node

SET CURRENT := NEXT.

[End of while Loop]

3. [*Free the Main Structure*]

CALL free(LST).

4. [*Success Message*]

Write: "Memory successfully freed. Skip List destroyed."

5. [*Exit*]

END OF ALGORITHM.
